Lab 2: Lập trình C cơ bản
(Basic C Programming on Linux)

Hướng dẫn thực hành Lập trình hệ thống

Trường Đại học Công nghệ - ĐHQGHN

Ngày 25 tháng 9 năm 2026

Mục lục

1. Mở đầu: từ shell sang C
2. Viết, biên dịch và chạy trên Linux
3. Cấu trúc một chương trình C
4. Biến và kiểu dữ liệu
5. Toán tử và thao tác bit
6. Cấu trúc điều khiển
7. Mảng
8. Xâu ký tự và string.h
9. Nhập/xuất chuẩn stdio.h
10. Gỡ lỗi và tra cứu
11. Bài tập thực hành
12. Tổng kết

Nội dung

1. Mở đầu: từ shell sang C
2. Viết, biên dịch và chạy trên Linux
3. Cấu trúc một chương trình C
4. Biến và kiểu dữ liệu
5. Toán tử và thao tác bit
6. Cấu trúc điều khiển
7. Mảng
8. Xâu ký tự và string.h
9. Nhập/xuất chuẩn stdio.h
10. Gỡ lỗi và tra cứu
11. Bài tập thực hành
12. Tổng kết

Mục tiêu buổi thực hành

Sau buổi hôm nay, sinh viên phải làm được

1. **Viết – biên dịch – chạy:** soạn file `.c` bằng `vim`, biên dịch bằng `gcc` có bật cảnh báo, chạy và đọc mã trả về.
2. **Dữ liệu và toán tử:** chọn đúng kiểu, hiểu chia nguyên, tràn số, và thao tác được *từng bit*.
3. **Điều khiển luồng:** dùng `if/else`, `switch`, `for`, `while` để giải bài toán có điều kiện và lặp.
4. **Mảng, chuỗi, nhập/xuất:** xử lý dãy số và văn bản bằng mảng, `string.h`, `printf/scanf/fgets`.

Sản phẩm cần nộp

14 chương trình `bai01.c` ... `bai14.c`: 1 bài biến, 2 bài bitwise, 4 bài điều khiển, 4 bài mảng, 3 bài chuỗi. Mọi file phải biên dịch với `-Wall -Wextra` **không còn cảnh báo**.

C là ngôn ngữ "bản địa" của hệ điều hành

Kernel Linux, `glibc`, `bash`, `gcc` đều viết bằng C. Giao diện system call *là* giao diện hàm C.

Học ở Lab 2	**Gặp lại ở đâu**	**Ví dụ**
`return 0` trong `main`	`$?` của shell (Lab 1)	`./a.out; echo $?`
`printf` ra màn hình	System call `write`	`write(1, buf, n)`
`scanf`/`fgets` từ bàn phím	fd 0 = `stdin`	`./prog < input.txt`
Toán tử bit `|`, `&`	Cờ khi mở file, quyền truy cập	`O_WRONLY|O_CREAT`, `0644`
Mảng `char buf[256]`	Bộ đệm đọc/ghi	`read(fd, buf, sizeof buf)`
Chuỗi kết thúc `'\0'`	Đường dẫn, argv, biến môi trường	`open("/etc/passwd", ...)`

Thông điệp chính

Lab 2 trông như "C nhập môn", nhưng mỗi thói quen hôm nay (kiểm tra kích thước mảng, kiểm tra giá trị trả về, hiểu từng bit) là điều kiện sống còn khi gọi system call ở các tuần sau.

Nội dung

1. Mở đầu: từ shell sang C
2. Viết, biên dịch và chạy trên Linux
3. Cấu trúc một chương trình C
4. Biến và kiểu dữ liệu
5. Toán tử và thao tác bit
6. Cấu trúc điều khiển
7. Mảng
8. Xâu ký tự và string.h
9. Nhập/xuất chuẩn stdio.h
10. Gỡ lỗi và tra cứu
11. Bài tập thực hành
12. Tổng kết

Chu trình soạn – biên dịch – chạy

```
gcc --version                          # kiem tra da co gcc (neu chua: sudo apt install build-
    essential)
vim hello.c                            # soan: i -> go code -> Esc -> :wq
gcc -Wall -Wextra -std=c11 -g hello.c -o hello
./hello                                # -> Hello, World!
echo $?                                # -> 0  (gia tri return cua main)
```

Tài liệu Lab đặt mục này ở *cuối* (mục 7) – hãy làm nó *đầu tiên*

Không biên dịch được thì không kiểm chứng được bất kỳ kiến thức nào ở các mục 1–6. Mở hai terminal: một cho `vim`, một cho `gcc` và chạy thử.

Vì sao phải gõ `./hello` chứ không phải `hello`?

Giống hệt script ở Lab 1: thư mục hiện tại không nằm trong `$PATH`. Thiếu `-o hello` thì gcc đặt tên mặc định là `a.out`.

Bốn giai đoạn bên trong một lệnh gcc

Giai đoạn	Cờ dừng tại đó	Đầu ra	Việc được làm
1. Tiền xử lý	`gcc -E hello.c`	`hello.i`	Chép `stdio.h` vào, thay macro
2. Biên dịch	`gcc -S hello.c`	`hello.s`	C → hợp ngữ (assembly)
3. Hợp dịch	`gcc -c hello.c`	`hello.o`	Hợp ngữ → mã máy, *chưa* chạy được
4. Liên kết	`gcc hello.o -o hello`	`hello`	Nối với `libc` (chứa `printf`)

Thử ngay

`gcc -E hello.c | wc -l` → vài trăm dòng: `#include` thực chất là *chép và dán* văn bản. `file hello.o hello` → "relocatable" so với "executable".

Đọc được lỗi thuộc giai đoạn nào là sửa được một nửa

Lỗi cú pháp → giai đoạn 2. "`undefined reference to ...`" → giai đoạn 4 (thiếu hàm lúc liên kết).

Cờ biên dịch nên dùng mọi lúc

Cờ	Tác dụng
`-Wall -Wextra`	Bật phần lớn cảnh báo: biến chưa khởi tạo, sai định dạng, `if (x = 0)`
`-std=c11`	Chuẩn C11 (`for (int i = 0; ...)` cần từ C99)
`-g`	Nhúng thông tin gỡ lỗi để dùng gdb
`-o ten`	Đặt tên file thực thi
`-fsanitize=address,undefined`	Bắt truy cập ngoài mảng, tràn số có dấu *lúc chạy*
`-Werror`	Coi cảnh báo là lỗi – dùng khi chấm bài

Cảnh báo không phải "lỗi nhẹ"

Với C, *đa số* bug của người mới đều đã được gcc cảnh báo trước – chỉ là không ai đọc. Quy tắc của môn: **một cảnh báo còn tồn tại = bài chưa xong**.

Đọc thông báo lỗi của gcc

Định dạng: `file:dòng:cột: error|warning: mô tả`. **Luôn sửa lỗi *đầu tiên* trước** – các lỗi sau thường là hệ quả.

Thông báo	**Nguyên nhân thường gặp**
`expected ';' before ...`	Thiếu dấu ; ở *dòng phía trên*
`implicit declaration of function 'printf'`	Quên `#include <stdio.h>`
`'i' undeclared`	Gõ sai tên biến hoặc dùng ngoài phạm vi { }
`stray '\342' in program`	Dấu nháy cong “ ” chép từ PDF/Word
`format '%d' expects ... 'int *'`	Quên & trong scanf
`undefined reference to 'main'`	Gõ sai `mian`, hoặc biên dịch nhầm file

Lỗi `stray '\342'` sẽ gặp ngay hôm nay

Tài liệu Lab in `printf` với *nháy kép cong*. Copy–paste từ PDF ⇒ gcc không biên dịch được. **Gõ tay**, và nếu đã dán thì thay lại bằng nháy thẳng ".

Nội dung

1. Mở đầu: từ shell sang C
2. Viết, biên dịch và chạy trên Linux
3. Cấu trúc một chương trình C
4. Biến và kiểu dữ liệu
5. Toán tử và thao tác bit
6. Cấu trúc điều khiển
7. Mảng
8. Xâu ký tự và string.h
9. Nhập/xuất chuẩn stdio.h
10. Gỡ lỗi và tra cứu
11. Bài tập thực hành
12. Tổng kết

Giải phẫu `hello.c`

```c
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

- `#include` – chỉ thị *tiền xử lý*: chép khai báo của `printf` vào.
- `int main(void)` – điểm bắt đầu; kiểu trả về `int`.
- `{ ... }` – khối lệnh; mỗi câu lệnh kết thúc bằng `;`.
- `"\n"` – *một* ký tự xuống dòng.
- `return 0;` – mã trả về cho tiến trình cha (shell).

Hai chi tiết nhỏ nhưng hay bị hỏi

`<stdio.h>` tìm trong thư mục hệ thống `/usr/include`; `"my.h"` tìm cạnh file nguồn trước.
Escape sequence: `\n` xuống dòng, `\t` tab, `\\`, `\"`, `\0` (null); trong `printf` viết `%%` để in dấu `%`.

main trả về cho ai? – nối với $? của Lab 1

```c
/* exitcode.c */
#include <stdio.h>
int main(void) {
    printf("Xong\n");
    return 3;
}
```

```
$ gcc exitcode.c -o exitcode
$ ./exitcode ; echo $?
Xong
3
$ ./exitcode && echo "thanh cong"
Xong             # khong in "thanh cong"
```

Quy ước chung của C, shell và system call

0 = thành công, khác 0 = mã lỗi (0–255). Shell nhận giá trị này qua `wait()` rồi đặt vào `$?`. Trong bài tập: `return 1;` khi dữ liệu nhập sai, `return 0;` khi chạy xong bình thường.

Ngược với cảm giác thông thường

Trong *điều kiện* `if` thì 0 là *sai*; nhưng *mã trả về của chương trình* thì 0 là *thành công*.

printf là hàm thư viện – write mới là system call

```
$ strace -e trace=write ./hello
write(1, "Hello, World!\n", 14)         = 14
+++ exited with 0 +++
```

- `printf` (thư viện C, `man 3 printf`) định dạng chuỗi trong user space, rồi gọi `write` (system call, `man 2 write`) để nhờ kernel ghi ra **fd 1**.
- 14 là số byte: 13 ký tự nhìn thấy + 1 ký tự `'\n'`.
- `exited with 0` chính là `return 0` ở slide trước.

Liên hệ: mục man ở Lab 1

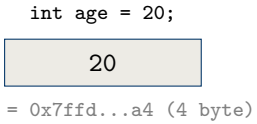
Mục 2 = system call, mục 3 = hàm thư viện C. Từ tuần này tra `man 3 printf`, `man 3 strlen`, `man 3 scanf`.

Nội dung

1. Mở đầu: từ shell sang C
2. Viết, biên dịch và chạy trên Linux
3. Cấu trúc một chương trình C
4. Biến và kiểu dữ liệu
5. Toán tử và thao tác bit
6. Cấu trúc điều khiển
7. Mảng
8. Xâu ký tự và string.h
9. Nhập/xuất chuẩn stdio.h
10. Gỡ lỗi và tra cứu
11. Bài tập thực hành
12. Tổng kết

Biến = tên + kiểu + vùng nhớ

- **Tên**: để người viết mã gọi.
- **Kiểu**: quyết định *bao nhiêu byte* và *cách hiểu các bit*.
- **Vùng nhớ**: có địa chỉ, lấy bằng `&age` – đây là lý do `scanf` cần `&`.



Quy tắc đặt tên (bắt buộc)

Chỉ gồm chữ cái `a--z` `A--Z`, chữ số `0--9`, dấu `_`; không bắt đầu bằng chữ số; không trùng từ khoá (`int`, `if`, `return` ...); **phân biệt hoa/thường** (`Age` $\neq$ `age`).

Quy ước (nên theo)

Tên có nghĩa: `score`, `num_students` thay vì `a`, `b1`. Biến đếm vòng lặp mới dùng `i`, `j`, `k`.

Các kiểu dữ liệu cơ bản (Linux x86-64)

Kiểu	Byte	Miền giá trị / độ chính xác	printf / scanf
`char`	1	$-128 \dots 127$ (một ký tự ASCII)	`%c`
`unsigned char`	1	$0 \dots 255$	`%hhu`
`int`	4	$\approx \pm 2{,}1 \times 10^9$	`%d`
`unsigned int`	4	$0 \dots 4{,}29 \times 10^9$	`%u`
`long`, `long long`	8	$\approx \pm 9{,}2 \times 10^{18}$	`%ld`, `%lld`
`float`	4	khoảng 6–7 chữ số có nghĩa	`%f` / `%f`
`double`	8	khoảng 15–16 chữ số có nghĩa	`%f` / `%lf`

Kích thước không phải hằng số của vũ trụ

Chuẩn C chỉ quy định *tối thiểu*: `long` 8 byte trên Linux 64-bit, 4 byte trên Windows. Kiểm tra:

```
printf("%zu\n", sizeof(long));
```

Tài liệu Lab nêu "kiểu dữ liệu" trong mục tiêu nhưng không có mục nào

Bài 1 (chiều cao), Bài 6 (đảo số), Bài 8 (tích) phụ thuộc trực tiếp vào bảng này.

Biến chưa khởi tạo chứa "rác"

```c
int sum;                    /* SAI: gia tri ban dau KHONG xac dinh */
for (int i = 0; i < n; i++)
    sum += a[i];            /* ket qua ngau nhien, co khi "tinh co" dung */

int sum = 0;                /* DUNG: luon khoi tao khi khai bao */
```

Vì sao lỗi này nguy hiểm

Biến cục bộ nằm trên *stack*, mang giá trị còn sót của lần dùng trước. Chạy thử một lần thấy đúng không chứng minh gì: đổi máy, đổi cờ `-O2` là kết quả khác. `-Wall -O2` cảnh báo `'sum' is used uninitialized`; ở `-O0` gcc có thể *im lặng*.

Giá trị khởi tạo đúng phụ thuộc bài toán

Tổng: `0`. Tích: `1` (Bài 8). Min/max: *phần tử đầu tiên* `a[0]`, không phải `0` (Bài 10).

char thực chất là một số nguyên nhỏ

```c
char c = 'A';
printf("%c %d\n", c, c);          /* -> A 65                          */
printf("%d\n", 'a' - 'A');        /* -> 32: khoang cach hoa-thuong    */
printf("%d\n", '7' - '0');        /* -> 7 : ky tu chu so -> gia tri   */
if (c >= 'A' && c <= 'Z') ...     /* kiem tra chu hoa (Bai 12)        */
```

Nhóm ký tự	`'0'` ... `'9'`	`'A'` ... `'Z'`	`'a'` ... `'z'`
Mã ASCII	48 ... 57	65 ... 90	97 ... 122

`'A'` khác `"A"`

`'A'` là *một* số nguyên (65). `"A"` là *mảng hai byte* `{'A', '\0'}`. Tra bảng đầy đủ: `man 7 ascii`.

Chia nguyên và ép kiểu

```c
printf("%d\n", 7 / 2);              /* -> 3  (bo phan le, KHONG lam tron)  */
float c = 5 / 9 * (f - 32);         /* SAI : 5/9 = 0 -> c luon bang 0      */
float c = 5.0 / 9.0 * (f - 32);     /* DUNG                                */
double avg = sum / n;               /* SAI : chia nguyen TRUOC, doi kieu SAU */
double avg = (double)sum / n;       /* DUNG: ep kieu TRUOC khi chia (Bai 10) */
```

Hai toán hạng kiểu `int` cho kết quả kiểu `int`

Kiểu của biến *nhận* kết quả không cứu được phép tính đã xong. Chỉ cần *một* toán hạng là `double` thì cả phép chia là chia thực.

Đúng cái bẫy đã gặp ở Lab 1

Shell: `echo $(( 7 / 2 ))` cho 3. C hành xử y hệt – và SP02 nhắc "dùng `9.0` chứ không phải 9".

Tràn số

```c
int big = 100000 * 100000;          /* TRAN SO: 1e10 vuot INT_MAX (2.1 ty)    */
long long bad = 100000 * 100000;    /* VAN SAI: nhan kieu int TRUOC, gan SAU  */
long long ok = 100000LL * 100000;   /* DUNG: hau to LL -> tinh bang 64 bit    */
unsigned char u = 255;  u++;        /* u == 0: so khong dau quay vong         */
```

Tràn số *có dấu* là hành vi không xác định (undefined behavior)

`INT_MAX + 1` thường ra số âm, nhưng chuẩn C không đảm bảo gì cả – trình biên dịch được phép giả định nó không bao giờ xảy ra. Số *không dấu* thì quay vòng theo modulo 2^n, có định nghĩa rõ ràng.

Công cụ

`#include <limits.h>`: `INT_MAX`, `LLONG_MAX`. `gcc -fsanitize=undefined` báo `signed integer overflow` kèm số dòng. Nhớ Fibonacci ở Lab 1: $F(94)$ tràn số 64 bit của bash – C cũng vậy.

Nội dung

1. Mở đầu: từ shell sang C
2. Viết, biên dịch và chạy trên Linux
3. Cấu trúc một chương trình C
4. Biến và kiểu dữ liệu
5. Toán tử và thao tác bit
6. Cấu trúc điều khiển
7. Mảng
8. Xâu ký tự và string.h
9. Nhập/xuất chuẩn stdio.h
10. Gỡ lỗi và tra cứu
11. Bài tập thực hành
12. Tổng kết

Toán tử số học, quan hệ, logic

```
7 / 2  == 3    -7 / 2 == -3
7 % 2  == 1    -7 % 2 == -1
x++   /* dung x ROI moi tang */
++x   /* tang ROI moi dung   */
```

% chỉ dùng cho số nguyên; kết quả mang **dấu của số bị chia**.

```
(a > b)        /* 1 neu dung, 0 neu sai */
(x >= 18) && (x <= 65)
(n == 0) || (m % n == 0)  /* an toan */
!found
```

Short-circuit: && dừng khi vế trái sai; || dừng khi vế trái **đúng**.

= là gán, == là so sánh

`if (x = 0)` *gán* 0 cho x: điều kiện luôn sai và x mất giá trị. `-Wall`: `suggest parentheses around assignment used as truth value`.

Hệ quả của -7 % 2 == -1 – gặp lại ở Bài 9

Kiểm tra số lẻ bằng `a[i] % 2 == 1` *bỏ sót mọi số lẻ âm*. Viết `a[i] % 2 != 0`.

Toán tử bit: làm việc trên từng bit

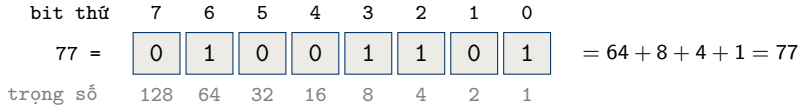
Tài liệu Lab **không có mục nào** về toán tử bit – trong khi Bài 2 và Bài 3 đều là bài bitwise.

Toán tử	Tên	Quy tắc	Ví dụ (8 bit)
`&`	AND	1 nếu *cả hai* bit là 1	`0101 & 0011 = 0001`
`|`	OR	1 nếu *ít nhất một* bit là 1	`0101 | 0011 = 0111`
`^`	XOR	1 nếu hai bit *khác nhau*	`0101 ^ 0011 = 0110`
`~`	NOT	Đảo mọi bit	`~00000101 = 11111010`
`<<`	Dịch trái	Nhân 2 mỗi bước	`00000001 << 3 = 00001000`
`>>`	Dịch phải	Chia 2 mỗi bước	`01001101 >> 2 = 00010011`

Bit khác logic

`5 & 3` $= 1$ nhưng `5 && 3` $= 1$ vì một lý do khác hẳn. `6 & 1` $= 0$ nhưng `6 && 1` $= 1$. `~5` $= -6$ với `int` 32 bit, không phải 1010.

Đánh số bit: bit 0 là bit thấp nhất (LSB)



- Dịch `x` sang phải `k` bước đưa bit thứ `k` về vị trí 0.
- AND với 1 xoá mọi bit khác, chỉ giữ lại bit 0.

Công thức lấy bit thứ `k` – lời giải cốt lõi của Bài 2 và Bài 3

```
(x >> k) & 1
```

Ví dụ `x = 10 = 1010`, `k = 1`: `1010 >> 1 = 0101`, `0101 & 0001 = 1`.

Bốn thao tác bit kinh điển

Mục đích	Biểu thức	Giải thích
Đọc bit k	`(x >> k) & 1`	Kết quả 0 hoặc 1
Bật bit k	`x | (1u << k)`	OR với mặt nạ chỉ có bit k
Tắt bit k	`x & ~(1u << k)`	AND với mặt nạ có mọi bit trừ bit k
Đảo bit k	`x ^ (1u << k)`	XOR với 1 thì đảo, với 0 thì giữ nguyên
Kiểm tra chẵn	`(x & 1) == 0`	Bit 0 quyết định chẵn/lẻ

Đây là cách hệ điều hành nói chuyện với bạn

`open("log.txt", O_WRONLY | O_CREAT | O_APPEND, 0644)`: mỗi cờ là *một bit*, ghép bằng `|`. `0644` là hằng số **bát phân** – đúng quyền `rw-r--r--` của `chmod` ở Lab 1.

Hằng số bắt đầu bằng 0 là số bát phân

`int x = 010;` → `x == 8`, không phải 10. `0x` là thập lục phân: `0xFF == 255`.

Độ ưu tiên toán tử – cứ nghi ngờ là thêm ngoặc

```c
if (x & 1 == 0)      /* SAI: la x & (1 == 0) = x & 0 = 0, luon sai      */
if ((x & 1) == 0)    /* DUNG                                             */
y = 1 << k + 1;      /* la 1 << (k + 1), vi + uu tien hon <<             */
```

Mức (cao → thấp)	Toán tử
1	() [] ++ -- (hậu tố)
2	! ~ ++ -- (tiền tố) (kiểu) sizeof
3	* / % rồi + - rồi << >>
4	< <= > >= rồi == !=
5	& rồi ^ rồi | **(thấp hơn ==!)**
6	&& rồi || rồi ?: rồi = += -= ...

gcc -Wall bắt được lỗi dòng 1

```
suggest parentheses around comparison in operand of '&'
```

Nội dung

1. Mở đầu: từ shell sang C
2. Viết, biên dịch và chạy trên Linux
3. Cấu trúc một chương trình C
4. Biến và kiểu dữ liệu
5. Toán tử và thao tác bit
6. Cấu trúc điều khiển
7. Mảng
8. Xâu ký tự và string.h
9. Nhập/xuất chuẩn stdio.h
10. Gỡ lỗi và tra cứu
11. Bài tập thực hành
12. Tổng kết

if / else if / else

```
if (score >= 90)      grade = 'A';
else if (score >= 80) grade = 'B';
else if (score >= 70) grade = 'C';
else if (score >= 60) grade = 'D';
else                  grade = 'F';
```

```
if (score >= 60)      grade = 'D';
else if (score >= 70) grade = 'C';
/* SAI: 95 da roi vao nhanh dau */
```

Chuỗi `else if` dừng ở điều kiện đúng *đầu tiên*

Nhờ vậy nhánh B không cần viết `score >= 80 && score < 90`: đã tới đây thì chắc chắn `score < 90`. Hệ quả: **sắp xếp điều kiện từ ngưỡng cao xuống thấp**. Tài liệu Lab chỉ có `if/else` hai nhánh – Bài 4 cần dạng này.

Luôn dùng ngoặc nhọn khi thân có hơn một lệnh

`if (x > 0) printf("duong\n"); count++;` – `count++` chạy *mọi lúc*, thụt lề không có nghĩa gì với C. Cũng vậy: `if (x > 0);` với dấu `;` thừa là một câu lệnh rỗng.

switch: khi nào dùng, khi nào không

```
switch (score / 10) {
    case 10:          /* roi xuong */
    case 9:  grade = 'A'; break;
    case 8:  grade = 'B'; break;
    case 7:  grade = 'C'; break;
    case 6:  grade = 'D'; break;
    default: grade = 'F';
}
```

- Biểu thức của switch phải là **số nguyên** (int, char); case phải là **hằng số**.
- Không so được khoảng (case >= 90), số thực, hay chuỗi.
- Mẹo score / 10 biến khoảng thành hằng số – chia nguyên có ích!

Quên break ⇒ chạy tràn xuống (fall-through)

Không có break, sau khi khớp case 3 chương trình *tiếp tục* chạy thân của case 4, default... Tài liệu Lab có break trong ví dụ nhưng không giải thích nó để làm gì. Chỉ bỏ break khi *cố ý*, và ghi chú rõ như case 10 ở trên.

Ba vòng lặp và khi nào dùng vòng nào

```c
for (int i = 1; i <= 10; i++)
    printf("%d\n", i);
```

Biết trước số lần lặp (Bài 5, duyệt mảng).

```c
while (n > 0) {
    d = n % 10;
    n /= 10;
}
```

Lặp tới khi điều kiện sai, không biết trước số lần (Bài 6).

```c
do {
    printf("n = ");
    scanf("%d", &n);
} while (n <= 0);
```

Thân chạy *ít nhất một lần* (nhập lại tới khi hợp lệ).

break và continue

`break`: thoát *vòng lặp gần nhất* ngay lập tức (Bài 7: đã tìm thấy ước thì dừng). `continue`: bỏ phần còn lại của lần lặp này, sang lần lặp kế. Tài liệu Lab thiếu `do-while`, `break`, `continue` – cả ba đều có trong SP02.

Vòng lặp vô hạn

Quên `x++` trong `while (x <= 5)` $\Rightarrow$ chương trình treo. Nhấn **Ctrl+C** – chính là tín hiệu `SIGINT` của Lab 1.

Dò vết (trace) một vòng lặp bằng tay

```c
int n = 1234, rev = 0;
while (n > 0) {
    rev = rev * 10 + n % 10;
    n = n / 10;
}
```

Lần	`n % 10`	`rev`	`n` (sau)
0	–	0	1234
1	4	4	123
2	3	43	12
3	2	432	1
4	1	4321	$0 \rightarrow$ dừng

Kỹ năng quan trọng nhất của buổi hôm nay

Trước khi chạy, **tự lập bảng giá trị** các biến qua từng vòng. Khi kết quả sai, so bảng của bạn với giá trị thật (in bằng `printf` hoặc xem bằng gdb) – chỗ lệch đầu tiên là chỗ có lỗi.

Nội dung

1 Mở đầu: từ shell sang C

2 Viết, biên dịch và chạy trên Linux

3 Cấu trúc một chương trình C

4 Biến và kiểu dữ liệu

5 Toán tử và thao tác bit

6 Cấu trúc điều khiển

7 Mảng

8 Xâu ký tự và string.h

9 Nhập/xuất chuẩn stdio.h

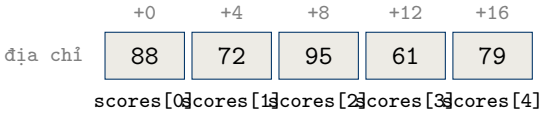
10 Gỡ lỗi và tra cứu

11 Bài tập thực hành

12 Tổng kết

Mảng: các phần tử cùng kiểu, nằm liên tiếp

```
int scores[5] = {88, 72, 95, 61, 79};
```



- Chỉ số bắt đầu từ **0**; phần tử cuối là `scores[4]`, *không có* `scores[5]`.
- Mỗi `int` chiếm 4 byte nên địa chỉ tăng 4: `&scores[i]` = `&scores[0]` + `4*i`. Truy cập theo chỉ số là $O(1)$.
- `int a[5] = {0};` → cả 5 phần tử bằng 0. `int a[5];` (cục bộ) → 5 giá trị rác.
- `sizeof scores / sizeof scores[0]` = $20/4 = 5$ phần tử.

C không kiểm tra biên mảng

```c
int a[5];
for (int i = 0; i <= 5; i++)   /* SAI: <= ghi vao a[5] -- ngoai mang */
    a[i] = 0;
```

Truy cập ngoài biên là hành vi không xác định

Không exception, không thông báo: chương trình có thể chạy "đúng", in sai, ghi đè biến khác, hoặc chết với `Segmentation fault`. Đây là gốc rễ của lỗ hổng *buffer overflow*.

```
$ gcc -g -fsanitize=address oob.c -o oob && ./oob
==12345==ERROR: AddressSanitizer: stack-buffer-overflow on address ...
WRITE of size 4 ... #0 in main oob.c:4
```

Thói quen: vòng lặp chuẩn `for (i = 0; i < n; i++)` – dấu <, không phải <=. Khi nghi ngờ, biên dịch lại với `-fsanitize=address` rồi chạy thử.

Mảng có kích thước nhập từ bàn phím

```c
/* Cach 1: VLA (C99) */
int n;
scanf("%d", &n);
if (n <= 0 || n > 1000) {
    fprintf(stderr, "n sai\n");
    return 1;
}
int a[n];   /* SAU khi kiem tra */
```

```c
/* Cach 2: kich thuoc toi da */
#define MAX_N 1000
int a[MAX_N];
int n;
scanf("%d", &n);
if (n <= 0 || n > MAX_N)
    return 1;
```

Bài 10 ghi `char names[n][20]` – đó là VLA

Hợp lệ từ C99, nhưng `n` phải được đọc *trước* dòng khai báo, và phải kiểm tra: `n` âm là hành vi không xác định; `n` quá lớn làm tràn stack (mặc định 8 MB, xem `ulimit -s`).

Khai báo `int a[n];` khi `n` chưa được gán

`n` đang chứa rác ⇒ mảng có kích thước ngẫu nhiên. Tuần sau sẽ học `malloc` để giải quyết triệt để.

Các mẫu duyệt mảng cần thuộc lòng

```
long long sum = 0, prod = 1;              /* tong: 0, tich: 1                    */
int imax = 0, imin = 0, count = 0;        /* chi so cua max/min: bat dau tu a[0] */
for (int i = 0; i < n; i++) {
    sum  += a[i];
    prod *= a[i];
    if (a[i] > a[imax]) imax = i;         /* luu CHI SO, lay duoc ca vi tri      */
    if (a[i] < a[imin]) imin = i;
    if (a[i] % 2 == 0) count++;           /* dem theo dieu kien                  */
}
double avg = (double)sum / n;
```

Tìm kiếm tuyến tính (SP02): độ phức tạp $O(n)$

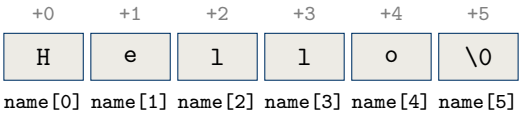
Duyệt từng phần tử và so sánh với giá trị cần tìm. Tốt nhất 1 phép so sánh, xấu nhất n phép. Bài 11 yêu cầu *mọi* vị trí, nên **không được** `break` ở lần gặp đầu tiên $\Rightarrow$ luôn đúng n phép so sánh.

Nội dung

1. Mở đầu: từ shell sang C
2. Viết, biên dịch và chạy trên Linux
3. Cấu trúc một chương trình C
4. Biến và kiểu dữ liệu
5. Toán tử và thao tác bit
6. Cấu trúc điều khiển
7. Mảng
8. Xâu ký tự và string.h
9. Nhập/xuất chuẩn stdio.h
10. Gỡ lỗi và tra cứu
11. Bài tập thực hành
12. Tổng kết

Chuỗi = mảng char kết thúc bằng '\0'

```c
char name[6] = "Hello";      /* 5 ky tu + 1 ky tu ket thuc */
```



- `char` chiếm **1 byte** ⇒ địa chỉ tăng 1 (không phải 4 như mảng `int`).
- `'\0'` (giá trị 0) là *cách duy nhất* để các hàm biết chuỗi kết thúc ở đâu – C không lưu độ dài.
- `char s[] = "Hello";` để trình biên dịch tự tính kích thước (6).

sizeof khác strlen

`char s[20] = "Hello";` → `sizeof s == 20` (dung lượng mảng), `strlen(s) == 5` (số ký tự trước `'\0'`).

Duyệt chuỗi và mảng chuỗi

```c
/* duyet den '\0', khong can strlen */
for (int i = 0; s[i] != '\0'; i++)
    if (s[i] == ' ') spaces++;
```

```c
char names[3][20] = {"Alice", "Bob", "Carol"};
printf("%s\n", names[1]);   /* Bob */
printf("%c\n", names[1][0]); /* B  */
```

Mảng hai chiều `names[3][20]`

3 dòng $\times$ 20 byte, nằm liên tiếp. `names[i]` là một chuỗi, `names[i][j]` là một ký tự. Mỗi dòng chứa tối đa **19** ký tự + `'\0'`.

Đính chính ví dụ ở mục 5.2 của tài liệu Lab

`printf("%c\n", name[0]); printf("%c\n", name[1]);` in ra **hai dòng** `H` và `e`, không phải `He` trên một dòng như tài liệu ghi.

Các hàm `string.h` và cái giá của chúng

Hàm	Chức năng	Điều phải nhớ
`strlen(s)`	Độ dài, không tính `'\0'`	Trả `size_t` → in bằng `%zu`
`strcpy(dst, src)`	Chép `src` vào `dst`	**Không kiểm tra** `dst` có đủ chỗ
`strcat(dst, src)`	Nối `src` vào cuối `dst`	**Không kiểm tra** `dst` có đủ chỗ
`strcmp(a, b)`	So sánh theo thứ tự từ điển	Trả **0 khi bằng nhau**
`strstr(s, sub)`	Tìm chuỗi con	Trả `NULL` nếu không có (Bài 14)
`strchr(s, c)`	Tìm ký tự	Trả `NULL` nếu không có
`strcspn(s, "\n")`	Vị trí ký tự đầu tiên thuộc tập	Dùng để xoá `'\n'` sau `fgets`
`snprintf(dst, n, ...)`	Ghi có định dạng, *có giới hạn*	Thay thế an toàn cho `strcpy`/`strcat`

Hai bẫy so sánh chuỗi

`if (a == b)` so sánh *địa chỉ*, không so sánh nội dung – luôn sai với hai mảng khác nhau. `if (strcmp(a, b))` nghĩa là "*khác nhau*" vì bằng nhau trả về 0. Viết rõ: `strcmp(a, b) == 0`.

ctype.h: phân loại ký tự

```c
#include <ctype.h>
unsigned char c = (unsigned char)s[i];   /* ep kieu truoc khi goi */
if (islower(c)) ...     /* 'a'..'z'                              */
if (isupper(c)) ...     /* 'A'..'Z'                              */
if (isdigit(c)) ...     /* '0'..'9'                              */
if (isspace(c)) ...     /* ' ', '\t', '\n', '\r', '\v', '\f'     */
c = tolower(c);         /* 'A' -> 'a', ky tu khac giu nguyen      */
```

Vì sao phải ép sang `unsigned char`?

`char` trên x86 là số có dấu. Chữ có dấu tiếng Việt trong UTF-8 gồm các byte $\geq 128 \rightarrow$ thành số âm; truyền số âm (khác `EOF`) cho `isdigit` là hành vi không xác định.

Hệ quả thú vị với UTF-8

`strlen("Việt") == 6`: chữ ệ chiếm 3 byte. Bài 12 với đầu vào tiếng Việt có dấu sẽ không đếm các chữ có dấu – C làm việc với *byte*, không phải với *ký tự* theo nghĩa con người hiểu.

Nội dung

1. Mở đầu: từ shell sang C
2. Viết, biên dịch và chạy trên Linux
3. Cấu trúc một chương trình C
4. Biến và kiểu dữ liệu
5. Toán tử và thao tác bit
6. Cấu trúc điều khiển
7. Mảng
8. Xâu ký tự và string.h
9. Nhập/xuất chuẩn stdio.h
10. Gỡ lỗi và tra cứu
11. Bài tập thực hành
12. Tổng kết

printf: đặc tả định dạng

Đặc tả	Kiểu	Ví dụ → kết quả
%d / %u	int / unsigned	printf("%d", -42) → -42
%lld / %llu	long / long long	printf("%lld", 10000000000LL)
%f / %.2f	float, double	printf("%.2f", 3.14159) → 3.14
%c / %s	char / chuỗi	printf("%s", name)
%x / %o	hex / bát phân	printf("%x", 255) → ff
%zu	size_t	printf("%zu", strlen(s))
%p	địa chỉ	printf("%p", (void*)&x)
%5d / %-10s	độ rộng / căn trái	căn cột bảng điểm Bài 10

Đặc tả sai kiểu là hành vi không xác định

printf("%d", 3.5) không "làm tròn" mà in ra số vô nghĩa. -Wall bắt được mọi trường hợp này. Tài liệu Lab dùng %lu cho strlen – chạy được trên Linux 64-bit nhưng đúng chuẩn là %zu.

scanf: ba quy tắc bắt buộc

```c
int age;  double h;  char name[20];
if (scanf("%d", &age) != 1) { fprintf(stderr, "Loi nhap\n"); return 1; }  /* 1 */
scanf("%lf", &h);          /* 2. double -> %lf   (float -> %f)            */
scanf("%19s", name);       /* 3. mang: KHONG co &; gioi han 19 ky tu + '\0' */
```

1. **Giá trị trả về** = số mục đọc thành công. Người dùng gõ abc thay vì số → trả 0, biến *không* được gán.
2. `printf` dùng `%f` cho cả `float` và `double`; `scanf` thì **không**: `double` bắt buộc `%lf`.
3. Biến thường cần `&` (địa chỉ); tên mảng *đã là* địa chỉ nên không có `&`.

`scanf("%s")` chỉ đọc *một từ*

Nhập `Nguyen Van An` → `name` chỉ là `"Nguyen"`, phần còn lại nằm chờ trong bộ đệm và làm hỏng lần đọc sau. Bài 1 (tên), Bài 10 (tên), Bài 13 (câu) đều cần đọc *cả dòng* ⇒ dùng `fgets`.

fgets: đọc cả dòng một cách an toàn

```c
char line[100];
if (fgets(line, sizeof line, stdin) != NULL) {
    line[strcspn(line, "\n")] = '\0';   /* fgets GIU LAI '\n' -> xoa di */
}
```

- Đọc tối đa `sizeof line - 1` ký tự, luôn thêm `'\0'` $\Rightarrow$ *không thể* tràn mảng.
- Giữ lại `'\n'` ở cuối nếu dòng đủ ngắn. Quên xoá là lỗi kinh điển của Bài 14.
- Trả `NULL` khi hết dữ liệu (người dùng nhấn Ctrl+D, hoặc file chuyển hướng đã hết).

`gets` đã bị *loại khỏi chuẩn C* từ C11

Tài liệu Lab ghi "hiện nay ít dùng do vấn đề an toàn" – chưa đủ mạnh. `gets` không biết kích thước mảng, nên *không có cách nào* dùng nó an toàn. glibc cảnh báo khi liên kết: `the `gets' function is dangerous and should not be used.`

Bẫy lớn nhất: '\n' còn sót sau scanf

```
scanf("%d", &age);      /* go: 20<Enter> */
fgets(name, 50, stdin); /* doc NGAY '\n' */
printf("[%s]\n", name); /* -> [ ] rong  */
```

`scanf("%d")` dừng *trước* `'\n'`. `fgets` gặp ngay `'\n'` đó và trả về một dòng rỗng – chương trình "bỏ qua" câu hỏi nhập tên.

Ba cách chữa

1. Đọc chuỗi **trước**, số **sau** (Bài 1).
2. Xả phần còn lại của dòng sau mỗi `scanf`: `int c; while ((c = getchar()) != '\n' && c != EOF);` (Bài 10)
3. Chỉ đọc bằng `fgets`, rồi tách số bằng `sscanf(line, "%d", &age)` – bền vững nhất.

`fflush(stdin)` *không* phải cách chữa: chuẩn C không định nghĩa `fflush` cho luồng vào.

stdin, stdout, stderr – fd 0, 1, 2 của Lab 1

```
printf("Ket qua: %d\n", x);                /* = fprintf(stdout, ...) -> fd 1 */
fprintf(stderr, "Loi: n phai > 0\n");     /* thong bao loi -> fd 2          */
```

```
echo 77 | ./bai02                    # cap du lieu qua pipe, khong can go tay
./bai10 < sv.txt > ketqua.txt        # doc tu file, ghi ket qua ra file
./bai05 < in.txt | diff - expected.txt && echo PASS    # kiem thu tu dong
```

Kiểm thử bằng chuyển hướng: chuẩn bị input một lần, chạy lại sau mỗi lần sửa. Thông báo lỗi ra `stderr` nên không lẫn vào file kết quả – đúng quy ước "dữ liệu ra stdout, thông báo ra stderr" của Lab 1.

stdout có bộ đệm

Ra terminal: đẩy ra khi gặp `'\n'`. Ra file: đẩy khi đầy bộ đệm (vài KB). Chương trình chết giữa chừng có thể *mất* các dòng `printf` cuối ⇒ in gỡ lỗi ra `stderr` (không đệm).

Nội dung

1. Mở đầu: từ shell sang C
2. Viết, biên dịch và chạy trên Linux
3. Cấu trúc một chương trình C
4. Biến và kiểu dữ liệu
5. Toán tử và thao tác bit
6. Cấu trúc điều khiển
7. Mảng
8. Xâu ký tự và string.h
9. Nhập/xuất chuẩn stdio.h
10. Gỡ lỗi và tra cứu
11. Bài tập thực hành
12. Tổng kết

Bộ công cụ gỡ lỗi C

Công cụ	**Bắt được lỗi gì**
`gcc -Wall -Wextra`	Biến chưa khởi tạo, sai định dạng, = thay ==, thiếu `#include`
`-fsanitize=address`	Truy cập ngoài mảng, ngoài chuỗi – *kèm số dòng*
`-fsanitize=undefined`	Tràn số có dấu, dịch bit quá độ rộng, chia cho 0
`fprintf(stderr, ...)`	In giá trị trung gian; không bị đệm, không lẫn vào output
`gdb ./prog`	Dừng tại dòng bất kỳ, xem giá trị biến, chạy từng bước
`strace ./prog`	Xem các system call chương trình gọi

Quy trình khi chương trình cho kết quả sai

(1) Đọc hết cảnh báo của gcc. (2) Chạy với input nhỏ nhất gây sai. (3) Lập bảng dò vết bằng tay. (4) So với giá trị thật từ gdb hoặc `fprintf(stderr, ...)`. (5) Sửa *một* chỗ rồi chạy lại.

gdb: tối thiểu cần biết

```
$ gcc -g -Wall bai06.c -o bai06
$ gdb ./bai06
(gdb) break 12          # dung o dong 12
(gdb) run
Nhap n (> 0): 1234
(gdb) print rev         # $1 = 0
(gdb) next              # chay 1 dong
(gdb) display n         # in n moi buoc
(gdb) info locals
(gdb) continue
(gdb) quit
```

Lệnh	Ý nghĩa
`b` (break)	Đặt điểm dừng
`r` (run)	Chạy chương trình
`n` (next)	Dòng kế, không vào hàm
`s` (step)	Dòng kế, *vào* hàm
`p` (print)	In giá trị biểu thức
`c` (continue)	Chạy tới điểm dừng kế
`bt`	Ngăn xếp lời gọi

Khi gặp `Segmentation fault`

Chạy trong gdb, chương trình dừng đúng dòng gây lỗi; gõ `bt` rồi `p i` để xem chỉ số đang vượt biên.

Tra cứu: man cho lập trình C

Lệnh	Nội dung
`man 3 printf`	Toàn bộ đặc tả định dạng, cờ độ rộng, độ chính xác
`man 3 scanf`	Giá trị trả về, `%lf`, `%[^\n]`
`man 3 fgets`	Hành vi với `'\n'` và `EOF`
`man 3 string`	Danh sách mọi hàm trong `string.h`
`man 3 isalpha`	Toàn bộ họ hàm `ctype.h`
`man 7 ascii`	Bảng mã ASCII: thập phân, bát phân, hex
`man gcc`	Mọi cờ biên dịch (rất dài – tìm bằng `/-Wall`)

Phần SYNOPSIS cho biết cần `#include` gì

Ví dụ `man 3 strlen` mở đầu bằng `#include <string.h>` và `size_t strlen(const char *s);` – đọc dòng đó là biết kiểu trả về và đặc tả in đúng (`%zu`).

Nội dung

1. Mở đầu: từ shell sang C
2. Viết, biên dịch và chạy trên Linux
3. Cấu trúc một chương trình C
4. Biến và kiểu dữ liệu
5. Toán tử và thao tác bit
6. Cấu trúc điều khiển
7. Mảng
8. Xâu ký tự và string.h
9. Nhập/xuất chuẩn stdio.h
10. Gỡ lỗi và tra cứu
11. Bài tập thực hành
12. Tổng kết

Kế hoạch thời gian và các điểm dừng

Thời lượng	Nội dung	Sinh viên làm
0–10’	Mục tiêu, chu trình `vim`–`gcc`–chạy	`hello.c`, `echo $?`, `strace`
10–25’	Kiểu dữ liệu, toán tử, thao tác bit	**Bài 1, 2, 3**
25–45’	Cấu trúc điều khiển, dò vết vòng lặp	**Bài 4, 5, 6, 7**
45–65’	Mảng, VLA, các mẫu duyệt mảng	**Bài 8, 9, 10, 11**
65–85’	Chuỗi, `string.h`, `fgets` và bộ đệm	**Bài 12, 13, 14**
85–90’	Gỡ lỗi, tổng kết, giao bài về nhà	Hoàn thiện bài còn dở

Quy tắc của buổi thực hành

Gõ tay, không copy–paste từ PDF (nháy cong!). Mỗi bài: biên dịch với `-Wall -Wextra` **sạch cảnh báo**, chạy với ví dụ của đề *và* ít nhất một trường hợp biên (0, số âm, chuỗi rỗng, nhiều khoảng trắng).

Bài 1 – Thông tin cá nhân (Variables)

```c
char name[50];  int age;  float height;
printf("Ho va ten    : ");
if (fgets(name, sizeof name, stdin) == NULL) return 1;
name[strcspn(name, "\n")] = '\0';           /* bo '\n' ma fgets giu lai */
printf("Tuoi         : ");
if (scanf("%d", &age) != 1 || age < 0) {
    fprintf(stderr, "Loi: tuoi phai la so nguyen khong am\n"); return 1;
}
printf("Chieu cao (m): ");
if (scanf("%f", &height) != 1 || height <= 0) {
    fprintf(stderr, "Loi: chieu cao phai la so thuc duong\n"); return 1;
}
printf("\n===== THONG TIN CA NHAN =====\n");
printf("%-10s: %s\n",     "Ho ten",    name);
printf("%-10s: %d\n",     "Tuoi",      age);
printf("%-10s: %.2f m\n", "Chieu cao", height);
```

Bài 1 – chạy thử và phân tích

```
$ gcc -Wall -Wextra -std=c11 bai01.c -o bai01 && ./bai01
Ho va ten    : Nguyen Van An
Tuoi         : 20
Chieu cao (m): 1.72

===== THONG TIN CA NHAN =====
Ho ten    : Nguyen Van An
Tuoi      : 20
Chieu cao : 1.72 m
```

Ba lựa chọn cần giải thích được

Đọc **tên trước** bằng `fgets`: tên có khoảng trắng, và tránh `'\n'` sót – chiều cao là `float` vì có phần thập phân – `%-10s` căn trái để các dấu `:` thẳng cột (đề yêu cầu "rõ ràng, dễ đọc").

Đọc tên bằng `scanf("%s", name)`

Chỉ lấy được `Nguyen`; `Van` bị `scanf("%d")` đọc như tuổi → thất bại, age chứa rác.

Bài 2 – Biểu diễn nhị phân 8 bit (Bitwise)

```c
int n;
printf("Nhap n (0-255): ");
if (scanf("%d", &n) != 1 || n < 0 || n > 255) {
    fprintf(stderr, "Loi: n phai la so nguyen trong [0, 255]\n");
    return 1;
}
for (int i = 7; i >= 0; i--)        /* in bit cao nhat (MSB) truoc */
    printf("%d", (n >> i) & 1);
printf("\n");
```

Kiểm chứng: 77 → 01001101 0 → 00000000 255 → 11111111 256 → báo lỗi.

Lỗi thường gặp

(1) Vòng lặp `i` từ 0 lên 7 → in *ngược* (10110010). (2) Dùng phép chia 2 lấy dư: đúng, nhưng bit ra theo thứ tự thấp → cao nên phải lưu vào mảng rồi in ngược – và đó không phải "bitwise" như đề yêu cầu. (3) In không đủ 8 chữ số với số nhỏ (5 → 101 thay vì 00000101).

Bài 3 – Giá trị bit thứ k (Bitwise)

```c
int x, k;
int width = (int)(sizeof(x) * 8);            /* 32 bit tren Linux x86-64 */
/* ... doc x, doc k, kiem tra 0 <= k < width ... */
unsigned int ux = (unsigned int)x;           /* dich tren so KHONG dau   */
int bit = (ux >> k) & 1u;
printf("Bit thu %d cua %d la %d\n", k, x, bit);
```

Kiểm chứng: x=10, k=1 $\rightarrow$ 1 x=10, k=0 $\rightarrow$ 0 x=-1, k=31 $\rightarrow$ 1 k=32 $\rightarrow$ báo lỗi.

Vì sao phải kiểm tra k và chuyển sang unsigned?

- Dịch một int 32 bit đi $\geq$ 32 hoặc số âm bước là *hành vi không xác định*.
- Dịch phải một số *âm* có dấu phụ thuộc trình biên dịch; với unsigned thì luôn xác định.
- Đổi int âm sang unsigned giữ nguyên mẫu bit bù hai $\Rightarrow$ vẫn đọc đúng bit của x.

Bài 4 – Xếp loại học lực (Control structure)

```c
int score;  char grade;
printf("Nhap diem (0-100): ");
if (scanf("%d", &score) != 1 || score < 0 || score > 100) {
    fprintf(stderr, "Loi: diem phai la so nguyen trong [0, 100]\n");
    return 1;
}
if (score >= 90)      grade = 'A';     /* xet tu nguong CAO xuong THAP */
else if (score >= 80) grade = 'B';
else if (score >= 70) grade = 'C';
else if (score >= 60) grade = 'D';
else                  grade = 'F';
printf("Diem %d -> loai %c\n", score, grade);
```

Kiểm chứng tại *mọi ngưỡng*, không chỉ giữa khoảng

`100, 90, 89, 80, 79, 70, 69, 60, 59, 0` và hai giá trị sai `-1, 101`. Lỗi ở ngưỡng (`>` thay vì `>=`) chỉ lộ ra tại đúng các điểm này. Lời giải bằng `switch (score / 10)`: xem slide `switch`.

Bài 5 – Bảng cửu chương (Control structures)

```c
int n;
printf("Nhap n: ");
if (scanf("%d", &n) != 1) {
    fprintf(stderr, "Loi: n phai la so nguyen\n");
    return 1;
}
for (int i = 1; i <= 10; i++)
    printf("%d x %d = %d\n", n, i, n * i);
```

Bài dễ nhất – dùng để luyện kiểm thử tự động

`echo 5 | ./bai05 | tail -n 1` → `5 x 10 = 50`. `echo 5 | ./bai05 | grep -c ' = '` → 10 dòng.

Mở rộng

Căn cột bằng `%2d x %2d = %3d`; in cả bảng 2–9 bằng hai vòng `for` lồng nhau. `n = 300000000` thì `n * i` tràn `int` – đổi sang `long long`.

Bài 6 – Đảo ngược các chữ số (Control structures)

```c
int n;
printf("Nhap n (> 0): ");
if (scanf("%d", &n) != 1 || n <= 0) {
    fprintf(stderr, "Loi: n phai la so nguyen duong\n");
    return 1;
}
long long rev = 0;                  /* long long: 1999999999 -> 9999999991 */
while (n > 0) {
    rev = rev * 10 + n % 10;        /* lay chu so cuoi, gan vao ben phai rev */
    n /= 10;                        /* bo chu so cuoi */
}
printf("So dao nguoc: %lld\n", rev);
```

Kiểm chứng: $1234 \to 4321$ $7 \to 7$ $1999999999 \to 9999999991$ $1200 \to 21$ (!)

Bài 6 – hai câu hỏi phân tích

1200 nên ra 21 hay 0021?

Đề nói "đảo ngược thứ tự các chữ số" – cả hai cách hiểu đều hợp lý, *bài làm tốt phải nêu giả định*. Nếu muốn giữ số 0 đầu, in từng chữ số thay vì dựng lại một số:

```
do {
    printf("%d", n % 10);   /* 1200 -> in 0, 0, 2, 1 -> "0021" */
    n /= 10;
} while (n > 0);
```

Vì sao `rev` là `long long`?

`int` tối đa 2147483647. Đảo 1999999999 được 9999999991 – *vượt* miền `int`. Dùng `int rev` thì kết quả sai mà không có thông báo gì. Kiểm chứng bằng `-fsanitize=undefined`.

Bài 7 – Kiểm tra số nguyên tố (Control structure)

```c
int is_prime = (n >= 2);              /* 1 KHONG phai so nguyen to */
for (int i = 2; i <= n / i; i++) {    /* tuong duong i*i <= n, nhung khong tran so */
    if (n % i == 0) {
        is_prime = 0;
        break;                        /* da co uoc: dung ngay */
    }
}
printf("%d %s so nguyen to\n", n, is_prime ? "la" : "khong phai la");
```

Kiểm chứng: 1 → không 2 → là 9, 25 → không 97 → là 2147483647 → là (tức thì).

Vì sao chỉ cần thử tới $\sqrt{n}$?

Nếu $n = a \times b$ với $a \le b$ thì $a \le \sqrt{n}$: không có ước nào $\le \sqrt{n}$ thì không có ước nào cả. Với 2147483647: 46 340 phép chia thay vì hơn 2 tỷ – $O(\sqrt{n})$ so với $O(n)$.

`i * i <= n` tràn `int` khi `n` gần `INT_MAX`

`i <= n / i` tương đương về toán học nhưng không bao giờ tràn.

Bài 8 – Tích các phần tử mảng (Array)

```c
int n;
if (scanf("%d", &n) != 1 || n <= 0 || n > 1000) { /* bao loi */ return 1; }
int a[n];                          /* VLA: chi khai bao SAU khi da kiem tra n */
for (int i = 0; i < n; i++)
    if (scanf("%d", &a[i]) != 1) return 1;
long long product = 1;             /* phan tu don vi cua phep nhan la 1, khong phai 0 */
for (int i = 0; i < n; i++)
    product *= a[i];
printf("Tich = %lld\n", product);
```

Kiểm chứng: 1 2 3 4 $\rightarrow$ 24 5 0 $\rightarrow$ 0 100000 100000 100000 $\rightarrow$ 1000000000000000 (10^{15}).

Bài 8 – phân tích

Ba lỗi làm sai kết quả

1. `product = 0` → tích luôn bằng 0. Giá trị khởi tạo phải là *phần tử đơn vị* của phép toán.
2. `int product` → ba phần tử 100000 cho 10^{15}: tràn `int` ngay từ phép nhân thứ hai, không có thông báo nào.
3. Khai báo `int a[n];` *trước* `scanf("%d", &n)` → kích thước mảng là giá trị rác.

Ngay cả `long long` cũng có giới hạn

20 phần tử bằng 10 đã là $10^{20} > 9{,}2 \times 10^{18}$. Bài làm tốt *nêu rõ* giới hạn này thay vì im lặng in số sai.

Mở rộng

Gặp phần tử 0 thì có thể dừng sớm (`break`) – tích chắc chắn bằng 0. Muốn phát hiện tràn: `__builtin_mul_overflow` của gcc, hoặc so sánh với `LLONG_MAX / |a[i]|` trước khi nhân.

Bài 9 – Tách mảng chẵn và lẻ (Array)

```c
int a[n], even[n], odd[n];
int ne = 0, no = 0;                    /* so phan tu HIEN CO trong moi mang */
for (int i = 0; i < n; i++) {
    if (a[i] % 2 == 0) even[ne++] = a[i];   /* ghi vao vi tri ne, roi tang ne */
    else               odd[no++]  = a[i];
}
printf("Mang chan (%d):", ne);
for (int i = 0; i < ne; i++) printf(" %d", even[i]);   /* in ne phan tu, KHONG phai n */
printf("\nMang le   (%d):", no);
for (int i = 0; i < no; i++) printf(" %d", odd[i]);
printf("\n");
```

Kiểm chứng: `1 -3 4 0 7 -8` → chẵn `4 0 -8`, lẻ `1 -3 7`. `2 4 6` → mảng lẻ rỗng.

Hai lỗi kinh điển

(1) Điều kiện lẻ `a[i] % 2 == 1`: vì `-3 % 2 == -1` nên `-3` rơi vào *không mảng nào* (khi viết hai `if` riêng). (2) Dùng chung chỉ số `i` để ghi `even[i]` → mảng kết quả có "lỗ" chứa giá trị rác.

Bài 10 – Quản lý điểm sinh viên (Array)

```
int n, c;
if (scanf("%d", &n) != 1 || n <= 0 || n > 100) { /* bao loi */ return 1; }
while ((c = getchar()) != '\n' && c != EOF) ;           /* xa '\n' sau scanf        */
char names[n][20];  int scores[n];
for (int i = 0; i < n; i++) {
    printf("Ten sinh vien %d : ", i + 1);
    if (fgets(names[i], sizeof names[i], stdin) == NULL) return 1;
    size_t len = strcspn(names[i], "\n");
    if (names[i][len] == '\n') names[i][len] = '\0';    /* bo '\n' cuoi ten         */
    else while ((c = getchar()) != '\n' && c != EOF) ;  /* ten qua dai: bo phan thua */
    printf("Diem sinh vien %d: ", i + 1);
    if (scanf("%d", &scores[i]) != 1) return 1;
    while ((c = getchar()) != '\n' && c != EOF) ;
}
int sum = 0, imax = 0, imin = 0;                        /* luu CHI SO               */
printf("\n%-4s %-20s %5s\n", "STT", "Ho ten", "Diem");
for (int i = 0; i < n; i++) {
    printf("%-4d %-20s %5d\n", i + 1, names[i], scores[i]);
    sum += scores[i];
    if (scores[i] > scores[imax]) imax = i;
    if (scores[i] < scores[imin]) imin = i;
}
printf("Diem trung binh: %.2f\n", (double)sum / n);     /* ep kieu TRUOC khi chia   */
printf("Cao nhat       : %s (%d)\n", names[imax], scores[imax]);
printf("Thap nhat      : %s (%d)\n", names[imin], scores[imin]);
```

Bài 10 – những điểm cần giải thích được

Vì sao lưu *chỉ số* `imax` thay vì *giá trị* `max`?

Đề yêu cầu hiển thị *sinh viên* có điểm cao nhất – cần cả tên: có chỉ số thì lấy được `names[imax]` lẫn `scores[imax]`. Khởi tạo `imax = 0` (phần tử đầu), không phải `max = 0` – sai khi mọi điểm ≤ 0.

Vì sao cần xả `'\n'` ở *hai* chỗ?

Sau `scanf` đọc `n` và sau mỗi `scanf` đọc điểm đều còn `'\n'`; không xả thì `fgets` của sinh viên kế tiếp đọc ngay dòng rỗng – mất tên từ người thứ hai.

Tên dài hơn 19 ký tự

`fgets` chỉ lấy 19 ký tự, phần thừa *nằm lại* trong `stdin` và bị đọc như điểm $\rightarrow$ `scanf` thất bại. Nhánh `else` bỏ phần thừa đó. Kiểm chứng: nhập tên 30 ký tự; chạy `./bai10 < sv.txt` sau mỗi lần sửa.

Bài 11 – Tìm mọi vị trí của x (Linear search)

```c
int found = 0;                      /* co: da gap x lan nao chua */
for (int i = 0; i < n; i++) {       /* KHONG break: can MOI vi tri */
    if (a[i] == x) {
        printf("%d ", i);
        found = 1;
    }
}
if (!found) printf("Khong tim thay %d", x);
printf("\n");
```

Kiểm chứng: mảng `1 4 2 4 5`, `x = 4` → `1 3` `x = 9` → `Khong tim thay 9`.

So với `linearSearch` trong SP02

Hàm của SP02 trả về vị trí *đầu tiên* rồi dừng. Bài 11 cần *tất cả* nên bỏ điểm dừng sớm: luôn đúng n phép so sánh. Thiếu cờ `found` thì khi không tìm thấy chỉ in một dòng trống – người dùng không biết là "không có" hay "chương trình lỗi".

Bài 12 – Đếm chữ thường, chữ hoa, chữ số (String)

```c
char s[256];
if (fgets(s, sizeof s, stdin) == NULL) return 1;
int lower = 0, upper = 0, digit = 0;
for (int i = 0; s[i] != '\0'; i++) {          /* duyet den ky tu ket thuc '\0' */
    unsigned char c = (unsigned char)s[i];    /* ctype.h doi hoi gia tri khong am */
    if (islower(c))      lower++;
    else if (isupper(c)) upper++;
    else if (isdigit(c)) digit++;
}
```

Kiểm chứng: `Hello World 2024 ABC xyz` → thường 11, hoa 5, số 4. Không dùng `ctype.h` cũng được: `c >= 'a' && c <= 'z'` ... – đúng với ASCII.

Hai lỗi thường gặp

(1) Đọc bằng `scanf("%s")` → chỉ đếm từ đầu tiên. (2) Lặp tới `sizeof s` thay vì tới `'\0'` → đếm cả rác phía sau chuỗi.

Bài 13 – Đếm số từ (String)

```c
int count = 0, in_word = 0;          /* in_word: dang o BEN TRONG mot tu? */
for (int i = 0; s[i] != '\0'; i++) {
    if (isspace((unsigned char)s[i])) {
        in_word = 0;                 /* gap khoang trang: ra khoi tu */
    } else if (!in_word) {
        in_word = 1;                 /* khoang trang -> ky tu: BAT DAU mot tu moi */
        count++;
    }
}
```

Kiểm chứng: `"hoc lap trinh C"` → 4 `"  hoc  lap  trinh  "` → 3 dòng rỗng → 0 `"mot<TAB>tab"` → 2.

"Số từ = số khoảng trắng + 1" là sai

Sai với khoảng trắng liên tiếp, khoảng trắng đầu/cuối dòng, và chuỗi rỗng (ra 1 thay vì 0).

Tư duy máy trạng thái: chỉ đếm tại *bước chuyển* "ngoài từ" → "trong từ". Mẫu này dùng lại khi tự viết shell ở các tuần sau: tách dòng lệnh thành các đối số.

Bài 14 – Tìm chuỗi con (String)

```c
/* Cach 1: thu vien */
s[strcspn(s, "\n")] = '\0';
sub[strcspn(sub, "\n")] = '\0';
if (strstr(s, sub) != NULL)
    printf("Co xuat hien\n");
else
    printf("Khong xuat hien\n");
```

```c
/* Cach 2: tu cai dat, O(n*m) */
int n = strlen(s), m = strlen(sub);
int pos = -1;
for (int i = 0; i + m <= n; i++) {
    int j = 0;
    while (j < m && s[i+j] == sub[j])
        j++;
    if (j == m) { pos = i; break; }
}
```

Quên xoá `'\n'` – chương trình đúng *một cách tình cờ*

sub thành `"lap\n"` → không tìm thấy trong `"hoc lap trinh C\n"`. Nhưng với từ *cuối* `"C"` thì lại thấy (vì s cũng kết thúc bằng `"C\n"`) – kiểu lỗi nguy hiểm nhất.

Trường hợp biên: sub rỗng → luôn "có" (chuỗi rỗng là chuỗi con của mọi chuỗi); sub dài hơn s → vòng lặp không chạy; phân biệt hoa/thường: `"Lap"` $\notin$ `"hoc lap"`.

Nội dung

1. Mở đầu: từ shell sang C
2. Viết, biên dịch và chạy trên Linux
3. Cấu trúc một chương trình C
4. Biến và kiểu dữ liệu
5. Toán tử và thao tác bit
6. Cấu trúc điều khiển
7. Mảng
8. Xâu ký tự và string.h
9. Nhập/xuất chuẩn stdio.h
10. Gỡ lỗi và tra cứu
11. Bài tập thực hành
12. Tổng kết

Câu hỏi kiểm tra trong báo cáo

1. `return 0` trong `main` đi đâu, xem bằng cách nào?
2. `gcc -E`, `-S`, `-c` dừng ở giai đoạn nào?
3. Vì sao `(double)sum / n` khác `(double)(sum / n)`?
4. `-7 / 2` và `-7 % 2` bằng bao nhiêu?
5. Lấy bit thứ `k` của `x` thế nào? Vì sao nên dùng `unsigned`?
6. `x & 1 == 0` sai ở đâu?
7. Quên `break` trong `switch` thì sao?
8. Truy cập `a[5]` với `int a[5]` thì chuyện gì xảy ra?
9. `sizeof s` khác `strlen(s)` thế nào?
10. Vì sao `scanf` cần `&age` nhưng không cần `&name`?
11. Vì sao `fgets` sau `scanf` đọc được dòng rỗng?
12. Vì sao `if (strcmp(a, b))` nghĩa là "khác nhau"?

Tiêu chí chấm bài

Đúng đắn – Sạch cảnh báo `-Wall -Wextra` – Bền vững (kiểm tra `scanf` và miền giá trị, lỗi ra `stderr`, mã trả về) – Hiểu biết (giải thích lựa chọn kiểu, nêu giả định) – Kiểm chứng tại trường hợp biên.

Chuẩn bị cho tuần sau: Hàm, con trỏ, bộ nhớ động

Đọc trước

- K&R, *The C Programming Language*, Chương 4 (Functions) và Chương 5 (Pointers and Arrays)
- M. Kerrisk, *The Linux Programming Interface*, mục 6.3–6.6 (bộ nhớ tiến trình, stack, `argv`)

Luyện thêm nếu còn thời gian

- Biên dịch lại cả 14 bài với `-Werror -fsanitize=address,undefined` và chạy lại
- Viết `test.sh` (kỹ năng Lab 1): chạy mọi bài với file input, so kết quả bằng `diff`
- Viết lại Bài 2 theo ba cách: dịch bit, mặt nạ `0x80 >> i`, chia 2 lưu mảng

Câu hỏi dẫn sang tuần sau

Vì sao `scanf("%d", &age)` cần *địa chỉ* của age? Câu trả lời: **con trỏ** và cách C truyền tham số.

Cảm ơn đã lắng nghe!